

Security Analysis of Smart Contract Languages

Project in Cryptography and Information Security

Rúben Silva
Student: pg57900

Martim Redondo
Student: pg57889

May 18, 2026

Abstract

This report presents a reproducible security benchmark for Solidity contracts on the EVM, structured around the OWASP Smart Contract Top 10. Thirty Minimal Viable Exploit (MVE) contracts are organized as Vulnerable/Fixed pairs across the 10 categories, enabling independent measurement of true positives on Vulnerable contracts and false positives on Fixed variants.

Eleven tools spanning four paradigms are evaluated: static analysis (Slither, Aderyn, Semgrep, Solhint), symbolic and formal analysis (Mythril, Halmos), property-based fuzzing (Echidna, Medusa) and LLM-assisted review (LLaMA-3.3-70B, Gemini 2.5 Flash, Qwen3-32B). Fuzzing achieved the highest detection rate among general-purpose tools (10/15), with just one false positives on Fixed variants; Static tools showed moderate recall with high FP rates; Halmos produced formal proofs on the two categories with authored properties; LLM review completed all 30 contracts ($\Delta = +3.3$ for LLaMA).

This work establishes a version-pinned, tool-agnostic pipeline that quantifies the precision-recall trade-off across security paradigms. By using a controlled MVE baseline, the study demonstrates that no single paradigm is sufficient — the value lies in the strategic composition of the security pipeline.

1 Introduction

Smart contracts are core infrastructure for DeFi and other on-chain systems, but their security model offers zero tolerance for software faults because deployed code is immutable: exploits are irreversible and failures translate directly into financial loss [1], [2].

The OWASP Smart Contract Top 10 [3] provides a stable taxonomy for the most consequential failure categories, but the question of how reliably each detection paradigm covers each category — and at what precision cost — remains difficult to quantify. This report addresses that gap

on a small but controlled scale. The dataset consists of 30 Solidity contracts organized as Vulnerable/Fixed Minimal Viable Exploit (MVE) pairs, strictly aligned with the OWASP Smart Contract Top 10 [3]. This pairing makes both true positives (on Vulnerable contracts) and false positives (on Fixed contracts) directly measurable on the same code surface.

Eleven tools spanning four paradigms are evaluated against this baseline: static analysis, symbolic and formal analysis, property-based fuzzing, and LLM-assisted review. Section 2 reviews the technical landscape of smart-contract vulnerabilities; Section 3 motivates the choice of OWASP and Solidity/EVM; Section 4 details each of the ten OWASP categories; Section 5 defines the detection methodology and per-paradigm metric behavior; Section 6 presents the discussion; Section 7 concludes with practical recommendations.

2 State of the Art

The immutability of smart contracts dictates that post-deployment security flaws are irreversible [1]. Existing literature systematically categorizes these vulnerabilities across three architectural layers: the programming language (e.g., Solidity), the virtual machine execution environment (e.g., EVM) and the underlying blockchain protocol [4], [5]. However, comparative analysis within the field has historically been hindered by the absence of a standardized nomenclature, leading to identical mechanical flaws being classified under divergent terminologies (e.g., reentrancy is grouped with concurrency vulnerabilities in some frameworks and with control-flow vulnerabilities in others) [4].

At the language layer, arithmetic miscalculations and inadequate input validations continue to account for the majority of reported exploits [6]. Concurrency vulnerabilities (most notably Reentrancy) remain a critical threat vector, enabling attackers to hijack control flow via recursive calls before internal states are synchro-

nized [3], [6]. Furthermore, at the protocol layer, reliance on environmental variables such as `block.timestamp` for time-sensitive logic — including pseudo-random generation and freshness checks — exposes operations to bounded manipulation by block proposers [7].

Recent evidence demonstrates a strong correlation between a smart contract’s application domain and the typology of its exploited vulnerabilities [2]. Decentralized Finance (DeFi) applications, which manage immense liquidity and demand complex cross-contract interoperability, exhibit a disproportionately high prevalence of reentrancy attacks and business logic manipulation [8]. Simpler asset-emission contracts (such as standard ERC-20 tokens) historically show a different distribution skewed toward access control failures and basic arithmetic errors [2], though recent incidents (e.g., the 2025 Balancer V2 hack) demonstrate that access control remains a high-value vector across application categories.

Consequently, the OWASP Smart Contract Top 10 — alongside the SWC Registry — has become the standard reference for category-level classification of smart contract vulnerabilities, with OWASP receiving more recent maintenance updates and tighter alignment with current audit practice [3]. The remainder of this report operationalizes the OWASP taxonomy as a structured baseline for empirical tool evaluation. Section 3 motivates the empirical scope (Solidity/EVM); Section 4 presents the per-category technical analysis.

3 Foundational Choices

This chapter defines two decisions that structure the benchmark: the vulnerability taxonomy and the execution stack. Section 3.1 motivates OWASP Smart Contract Top 10 as the organizing taxonomy; Section 3.2 justifies restricting the empirical phase to Solidity on the EVM.

3.1 Why OWASP Smart Contract Top 10

The OWASP Smart Contract Top 10 is the current industry reference for on-chain risk categorization [3]. It is maintained by OWASP and periodically revised using real incident evidence, including major DeFi losses from 2024–2025.

Its adoption in this work is motivated by three properties. First, it provides a stable shared vocabulary already used in audits and

incident reports, allowing the benchmark to be read against findings reported in audit literature without translation. Second, its ten categories span language-level (Solidity-specific), EVM-level (bytecode-level) and protocol-level (consensus-dependent) failures. This vertical coverage matches the operational scope of the four detection paradigms evaluated in Section 5. Third, OWASP categories are scoped at the level of attack vector (e.g., price oracle manipulation) rather than implementation pattern (e.g., use of `block.timestamp`), which aligns with the audit-level granularity at which detection tools report findings.

Alternative references exist but each has a specific limitation: the SWC Registry covers only technical code-level flaws and excludes business-logic vectors and the DASP Top 10 has not received maintenance updates since 2018. OWASP spans both technical and business-logic layers and remains actively maintained, which makes it the most practical taxonomy for a category-level benchmark.

3.2 Why Solidity/EVM Stack

The empirical scope is deliberately restricted to Solidity contracts on the EVM for three reasons.

Ecosystem dominance. Ethereum and major EVM-compatible chains concentrate most DeFi TVL, and Solidity is the dominant production language in this environment [8]. A benchmark targeting practical relevance should therefore focus on this stack.

Taxonomic coverage. All OWASP categories can be instantiated directly in Solidity/EVM without semantic adaptation. Alternative stacks (e.g., Move on Sui, Rust/Wasm on Solana or NEAR) change exploit mechanics for some categories, which would require artificial mappings to remain category-aligned with OWASP.

Tooling maturity. Solidity is the only contract language where all four detection paradigms have production-grade tooling: semantic static analysis (Slither, Aderyn), symbolic execution (Mythril), formal verification (Halmos) and property-based fuzzing (Echidna, Medusa). Alternative stacks have partial coverage — Move has formal verification (Move Prover), but no mature fuzzer comparable to Echidna; Rust/Wasm has fuzzing harnesses, but no specialized static analyzer with EVM-level domain knowledge. A multi-paradigm benchmark therefore requires So-

lidity by construction.

Accordingly, the practical benchmark is restricted to Solidity/EVM. Cross-stack incidents are discussed only as contextual evidence in the theoretical framing.

4 Analysis of the OWASP Smart Contract Top 10

This section provides an in-depth technical analysis of the primary attack vectors identified in the OWASP Smart Contract Top 10 2026 [3]. To facilitate a comprehensive understanding, each vulnerability is deconstructed into its conceptual framework, EVM mechanics, and required mitigation strategies.

4.1 SC01:2026 - Access Control Vulnerabilities

Definition: Improper access control refers to situations where a smart contract fails to rigorously enforce who may invoke privileged behavior, under which conditions, and with which parameters. This encompasses weak trust boundaries across governance contracts, proxy admins, and core logic functions (e.g., token minting or pool reconfiguration).

Exploit Mechanics & Context: Attackers exploit missing modifiers (e.g., `onlyOwner`), unprotected initialization routines, or privilege confusion (such as incorrect assumptions of `msg.sender` via delegate calls or meta-transactions). By bypassing these checks, an untrusted address can impersonate a privileged actor or masquerade as a trusted protocol module. Recent exploits highlight the severity of these flaws: the Balancer V2 hack (\$128M loss in 2025) involved bypassing `msg.sender` checks in pool configurations, while the Cork Protocol exploit (\$11M loss in 2025) was caused by missing caller validation (`onlyPoolManager`) on Uniswap V4 hook callbacks.

Mitigation: Establish robust, explicit Role-Based Access Control (RBAC) utilizing battle-tested libraries (e.g., OpenZeppelin's `AccessControl`) rather than bespoke role systems. Privileges must be clearly separated (e.g., operational vs. emergency roles) and ideally held by multi-signature wallets or governance modules, avoiding reliance on single Externally Owned Accounts (EOAs) (as seen in the 2025 Zoth exploit). Furthermore, initialization

routines for upgradeable contracts must be securely locked (`_disableInitializers`), and all hook/callback entry points must explicitly validate the caller on-chain.

4.2 SC02:2026 - Business Logic Vulnerabilities

Conceptual Framework: Business logic vulnerabilities represent semantic, design-level flaws where a smart contract's intended economic or functional behavior is subverted. Unlike low-level bugs (e.g., reentrancy or integer overflows), the implementation is syntactically perfect and individual checks (such as type safety and access control) function exactly as written. However, the system's "rules of the game", its incentives, state transitions, and core invariants are modeled incorrectly, permitting exploitable and economically irrational outcomes.

Technical Anatomy & EVM Mechanics: The Ethereum Virtual Machine (EVM) strictly executes computational opcodes; it possesses no native understanding of economic concepts such as "fairness," "debt limits," or "staking rewards." As long as the mathematical operations do not underflow, divide by zero, or explicitly trigger a `revert()`, the EVM will finalize the transaction. Attackers exploit this absolute determinism by finding edge cases in path-dependent state machines or cross-module accounting.

For instance, an attacker might manipulate the order of operations (e.g., deposit, fail, claim, withdraw) to desynchronize internal accounting from external balances. This was notoriously demonstrated in the **Abracadabra money hack (\$12.9M loss in 2025)**, where attackers executed a three-stage exploit: they deliberately triggered a failed deposit to leave tokens stuck, initiated a self-liquidation that erased their position without removing the underlying order, and subsequently used this "ghost" collateral to borrow real assets.

Furthermore, complex algorithmic logic is highly susceptible to mathematical divergence. In the **Yearn Finance exploit (\$9M loss in 2025)**, attackers performed highly imbalanced add/remove liquidity operations that forced the pool's fixed-point iteration solver into a divergent regime. This caused the product term to collapse to zero, fundamentally converting a stable-swap invariant curve into a constant-sum curve, allowing the attacker to mint infinite LP tokens without adequate collateral.

Mitigation Strategies: Because static analysis tools verify syntax rather than economic semantics, mitigating business logic flaws requires a paradigm shift towards formal modeling.

- **Explicit Invariant Enforcement:** Core economic rules must be hard-coded and verified on every state transition (e.g., `require(debt[user] <= maxBorrow(user))`). Invariants such as "Total value withdrawn cannot exceed total deposits plus realized yield" must never be violable.
- **Advanced Testing:** Developers must utilize formal mathematical verification and property-based fuzzing to simulate millions of adversarial state transitions and path sequences.
- **Gated Deployments:** New strategies or modules must be rolled out behind strict liquidity caps and monitored closely before raising borrowing limits or integrating with the broader protocol ecosystem.

4.3 SC03:2026 - Price Oracle Manipulation

Conceptual Framework: Smart contracts cannot natively access off-chain data (such as fiat exchange rates or asset market prices) and must rely on Oracles to fetch this information. If a protocol trusts a single Decentralized Exchange (DEX) with low liquidity or fails to validate the freshness of the data, an attacker can artificially inflate or crash the asset's price momentarily. This behaviour can drastically overvalue an asset just before using it as collateral for a massive loan.

Technical Anatomy & EVM Mechanics: Automated Market Makers (AMMs) determine spot prices using formulas. The spot price is merely the ratio of reserves at an exact millisecond. An attacker can use a flash loan (see SC04) to dump a massive amount of Token A into a pool, artificially inflating the reserve of A and draining the reserve of B. The spot price of Token A crashes, while Token B skyrockets. The vulnerable protocol reads this distorted, intra-block spot price, allowing the attacker to use a minimal amount of Token B to borrow millions in stablecoins before the pool rebalances. This dynamic was a core component of the **NGP Token exploit (\$2M loss in 2025)**, where the protocol's `getPrice()` function relied solely on raw

DEX reserves, and the **GMX exploit (\$42M loss in 2025)**, where downward price manipulation of Bitcoin enabled the attacker to purchase GLP at artificially deflated prices.

Mitigation Strategies: Protocols must never rely on single, instantaneous on-chain spot prices.

- **Decentralized Aggregation:** Integrate robust oracle networks (e.g., Chainlink) and require multi-source aggregation (using medians) to reject outlier data.
- **Time-Weighted Average Prices (TWAP):** Utilize TWAPs over sufficient chronological windows to mathematically neutralize single-block, flash-liquidity manipulation.
- **Data Freshness Checks:** Implement strict validation checks on oracle responses (e.g., `require(block.timestamp - updatedAt <= MAX_DELAY)`) to prevent the ingestion of stale or stuck price data.
- **Circuit Breakers:** Halt sensitive operations (such as borrowing or liquidations) if anomalous deviations between the primary oracle and reference markets are detected.

4.4 SC04:2026 - Flash Loan-Facilitated Attacks

Conceptual Framework: Flash loans are a legitimate Decentralized Finance (DeFi) primitive that allows users to borrow entirely uncollateralized assets, provided the borrowed amount plus a nominal fee is returned within the exact same transaction block. Consequently, flash loans are not inherently software bugs; rather, they act as an asymmetric "force multiplier." They grant an attacker temporarily infinite capital, weaponizing otherwise microscopic logic (SC02), oracle (SC03), or arithmetic (SC07) vulnerabilities into catastrophic, protocol-draining exploits. Any system that assumes safety based on "capital at risk" or historical position sizes is critically exposed.

Technical Anatomy & EVM Mechanics: The Ethereum Virtual Machine (EVM) processes transactions atomically in a single-threaded execution context. This means a complex transaction comprising hundreds of sub-calls either finalizes entirely or, if a `require()` statement fails at any point (e.g., the loan cannot be repaid), the

entire state sequence reverts to its initial condition. Attackers leverage this to assume zero financial risk.

An attacker constructs a batched transaction payload: 1) Call a flash lender (e.g., Aave or Uniswap V3) to borrow millions of dollars. 2) Route these funds into a vulnerable protocol to aggressively skew an invariant. 3) Extract the artificially generated profit. 4) Repay the flash loan. Because this all occurs within the same `block.number`, historical protections fail.

This mechanism was the primary catalyst in the **Bunni exploit (\$8.4M loss in 2025)**, where an attacker flash-borrowed 3M USDT to push a pool's spot price to mathematical extremes (leaving an active USDC balance of merely 28 *wei*), and then executed 44 chained micro-withdrawals to systematically exploit a truncation error. Similarly, in the **zkLend exploit (\$9.5M loss in 2025)**, a fractional rounding-down error in the `mint()` function was continuously looped using flash-borrowed capital, inflating the lending accumulator to drain the protocol.

Mitigation Strategies: Protocols must be engineered under the strict paradigm that adversaries always possess infinite, transient capital.

- **Intra-Block Rate Limiting:** Implement strict checks (e.g., `require(lastUpdateBlock != block.number)`) to prevent the same user or contract from repeatedly executing high-impact state transitions (like deposits and withdrawals) within the same block lifecycle.
- **Exposure Caps:** Enforce maximum slip-page tolerances, strict borrowing caps, and maximum single-transaction position sizes to throttle the mathematical impact of a liquidity shock.
- **Adversarial Fuzzing:** QA testing and smart contract audits must natively integrate flash-loan simulations, utilizing property-based fuzzing to discover profitable multi-call execution sequences.
- **Root Cause Remediation:** Because flash loans merely amplify existing flaws, developers must rigorously harden the underlying price oracles and share-accounting arithmetic that make the arbitrage profitable in the first place.

4.5 SC05:2026 - Lack of Input Validation

Conceptual Framework: Lack of input validation occurs when a smart contract processes external data—such as function parameters, `calldata`, cross-chain messages, or signed payloads—without rigorously enforcing that the data is well-formed, authorized, and within expected mathematical bounds. It is the Web3 equivalent of a database accepting arbitrary SQL injection payloads or a system accepting a negative human age. Contracts that assume inputs are benign leave themselves open to adversarial data that pushes the system into unsafe states, corrupts accounting, or bypasses intended checks.

Technical Anatomy & EVM Mechanics: The Ethereum Virtual Machine (EVM) blindly relies on ABI (Application Binary Interface) encoding to parse and pass parameters into functions. It does not inherently know what a "safe fee" or a "valid address" is. If a contract fails to validate parameters—such as accepting a `0x0` address, an extreme numeric value, or an invalid cryptographic signature—the internal logic will execute using these corrupted values.

The catastrophic potential of unvalidated inputs is highlighted in the **Cetus exploit (\$223M loss in 2025)**. While the root cause involved a flawed overflow check, a critical contributing factor was the protocol's failure to enforce bounds on liquidity parameters, allowing the attacker to pass extreme values (e.g., $\sim 2^{113}$). When these unvalidated inputs interacted with the arithmetic logic, it produced edge cases that drained the pools. Similarly, in the **Ionic Money exploit (\$6.9M loss in 2025)**, attackers utilized social engineering to whitelist a counterfeit LBTC token. The protocol's on-chain logic failed to properly validate the authenticity of the listed token contract, allowing the attacker to deposit 250 fake LBTC and borrow millions in real assets.

Mitigation Strategies: Developers must adopt a "zero-trust" architecture for all external data, including inputs from protocol administrators and governance modules.

- **Strict Boundary Enforcement:** Hard-code explicit ranges for all numeric parameters (e.g., fees cannot exceed 10%, or 1,000 basis points) and ensure critical addresses are non-zero using `require()` statements or gas-efficient custom errors (e.g., `revert InvalidFee()`) at the start of execution.

- **Payload Verification:** Cryptographically verify all off-chain signed data, enforcing strict nonces, expirations, and `chainID` checks to prevent replay and ordering attacks across Layer-2 networks or bridges.
- **Negative Property Fuzzing:** Audit and QA processes must include property-based fuzzing specifically designed to bombard the contract with malformed, out-of-bounds, and unexpected `calldata` to ensure the contract safely reverts rather than entering an undefined state.

4.6 SC06:2026 - Unchecked External Calls

Conceptual Framework: An unchecked external call occurs when a smart contract invokes another contract or address without strictly validating the callee's behavior, the operation's return value, or the potential for malicious callbacks. It is the programmatic equivalent of executing a wire transfer, assuming the funds arrived successfully without verifying the bank receipt, and immediately updating the internal ledger. Furthermore, it implies a dangerous assumption of trust: assuming the external entity will simply receive the message and not execute arbitrary, hostile logic in return.

Technical Anatomy & EVM Mechanics: The Ethereum Virtual Machine (EVM) utilizes distinct mechanisms for inter-contract communication. When utilizing low-level opcodes (such as `call`, `delegatecall`, or `send`), the EVM does not automatically halt execution or bubble up `revert` errors if the receiving contract encounters a failure. Instead, it silently returns a boolean `false`. If a developer ignores this return value, the caller contract proceeds under the false assumption that the transfer succeeded, permanently desynchronizing its internal accounting from actual blockchain balances.

Additionally, whenever a contract makes an external call, it effectively hands over the execution control flow to the receiving address. If the receiving address is a smart contract, it can execute arbitrary logic before returning control. This was critically illustrated in the **GMX exploit (\$42M loss in 2025)**, where the `executeDecreaseOrder` function transferred funds to an attacker-supplied contract during a refund process. Because the state was updated *after* the call, the attacker leveraged the control-

flow handover to re-enter the protocol. Similarly, in the **Arcadia Finance hack (\$3.5M loss in 2025)**, the protocol allowed arbitrary external calls to user-supplied router addresses via `swapData` parameters without validating the callee. The attacker routed a callback through a malicious contract to spoof privileged execution contexts and drain the protocol.

Mitigation Strategies: Developers must adopt a strictly adversarial view of all external interactions; even "standard" tokens can be upgraded to include malicious hooks.

- **Explicit Return Validation:** Never ignore the boolean return values of low-level calls. For standard token transfers, universally adopt audited wrappers like OpenZeppelin's `SafeERC20`, which standardizes non-compliant token behaviors and forcefully reverts the transaction upon failure.
- **Checks-Effects-Interactions (CEI):** To mitigate the dangers of control-flow handover, internal state variables must be updated *before* the external call is dispatched.
- **Pull over Push:** Architect systems so that users independently "pull" (withdraw) their funds, rather than having the contract "push" funds to arbitrary addresses in batch loops, which opens vectors for Denial of Service (DoS) and callback manipulation.

4.7 SC07:2026 - Arithmetic Errors

Conceptual Framework: Arithmetic errors encompass flaws in integer math, scaling, or rounding operations that lead to precision loss or exploitable miscalculations. It is the programmatic equivalent of rounding down fractional cents on millions of banking transactions and silencing the difference.

Technical Anatomy & EVM Mechanics: The Ethereum Virtual Machine (EVM) fundamentally lacks native support for floating-point numbers (decimals). Consequently, all math is performed using integers. If division occurs before multiplication in a complex formula (e.g., $(A/B) \times C$), the EVM instantly truncates the decimal remainder of A/B to zero. Attackers execute repeated micro-transactions (often accelerated by flash loans) to force this truncation, incrementally bleeding value from interest rate accumulators or AMM pools.

This rounding vulnerability was central to the **zkLend exploit (\$9.5M loss in 2025)**, where an integer division in the `mint()` function rounded down. A withdrawal that should have burned 1.5 tokens was truncated to 1.0, allowing attackers to artificially inflate the lending accumulator via rapid, repeated deposits and withdrawals. Similarly, in the **Bunni exploit (\$8.4M loss in 2025)**, developers assumed rounding down an idle balance would inherently favor the protocol. However, repeated micro-operations allowed the attacker to decrease the pool's USDC balance by 85.7% while only burning 84.4% of their liquidity.

Mitigation Strategies: Mathematical operations must be strictly structured to multiply before dividing. Furthermore, developers must clearly document and mathematically prove their rounding strategy (e.g., ensuring rounding always favors the protocol's solvency, never the user). Complex financial logic should rely on audited fixed-point math libraries (such as `PRBMath`), and property-based fuzzing must be employed to discover edge cases around repeated fractional operations.

4.8 SC08:2026 - Reentrancy Attacks

Conceptual Framework: Reentrancy is a critical concurrency vulnerability where a smart contract performs an external call to an untrusted address, and the receiving entity maliciously "calls back" into the original contract before the initial execution thread has finished. Because the victim contract's internal state has not yet been updated, the attacker interacts with a "stale" state. It is analogous to an ATM dispensing cash and, before deducting the balance from the user's ledger, asking if they would like to perform another transaction. The user instantly requests more cash, looping the process until the machine's reserves are depleted.

Technical Anatomy & EVM Mechanics: The Ethereum Virtual Machine (EVM) operates on a synchronous, single-threaded execution model. When a smart contract executes an external call (e.g., transferring Ether via `msg.sender.call{value: amount}("")` or triggering an ERC-777/ERC-721 receiver hook), the control flow is immediately handed over to the recipient. If the recipient is a malicious smart contract, its `fallback()` or `receive()` function is automatically triggered.

Inside this fallback function, the attacker

recursively calls the victim's vulnerable function (e.g., `withdraw()`). Because the execution thread paused at the external call, the subsequent line of code—which deducts the user's balance—has not yet been executed. Consequently, the `require(balances[msg.sender] >= amount)` check passes repeatedly.

This vector extends beyond simple withdrawals. It can manifest as *Cross-Function* or *Cross-Contract* reentrancy, where attackers manipulate complex accounting. This was critically illustrated in the **GMX exploit (\$42M loss in 2025)**. The `executeDecreaseOrder` function transferred control to an attacker-supplied contract during a refund process. The attacker leveraged this execution handover to re-enter the protocol and manipulate global average short prices, Assets Under Management (AUM), and GLP valuations before the initial transaction finalized.

Mitigation Strategies: Defending against reentrancy requires strict architectural patterns to manage execution handovers.

- **Checks-Effects-Interactions (CEI) Pattern:** This is the fundamental defense. Developers must structure functions to first check preconditions (Checks), definitively update all internal state variables (Effects), and *only then* dispatch external calls (Interactions).
- **Mutex Guards:** Implement reentrancy locks, such as OpenZeppelin's `ReentrancyGuard` (`nonReentrant` modifier), which sets a boolean flag preventing a function from being executed again while it is already in the call stack.
- **Read-Only Reentrancy Awareness:** Protocols must ensure that `view` functions (like oracle price getters) cannot be accessed mid-transaction if the protocol's state is currently desynchronized, preventing third-party integrations from reading manipulated, intra-block data.

4.9 SC09:2026 - Integer Overflow and Underflow

Conceptual Framework: Integer overflow and underflow occur when an arithmetic operation produces a value that falls outside the permissible volumetric boundaries of its designated data type. It is a mechanical rollover flaw, analogous to an analog car odometer reaching 999,999 miles

and flipping back to 000,000. Conversely, an underflow allows a balance of 0 to subtract 1 and wrap backward to the maximum theoretical limit of the integer. When this occurs in financial logic, attackers can instantaneously transform zero balances into billions of tokens or bypass critical accounting invariants.

Technical Anatomy & System Mechanics: The mechanics of this vulnerability vary drastically depending on the virtual machine and compiler environment:

- **EVM / Solidity Context:** In Solidity versions prior to 0.8.0, integer wrapping occurred silently at the opcode level. Subtracting 1 from a `uint256` of 0 resulted in $2^{256} - 1$. While Solidity $\geq 0.8.0$ introduced native, default revert mechanisms for boundary breaches, developers often bypass this using explicit `unchecked` blocks for gas optimization. If an attacker can inject crafted inputs into an `unchecked` multiplication or addition chain, the silent wrap returns.
- **Non-EVM Context (Move/Sui):** On Rust-based or Move chains, default overflow semantics differ dangerously. For instance, in Move, while addition and multiplication will abort upon overflow, a *left bitwise shift* (`<<`) does not abort; it silently truncates the overflowing bits.

This cross-chain architectural nuance was the root cause of the devastating **Cetus Protocol exploit (\$223M loss in 2025)** on the Sui network. The protocol utilized a custom math library (`integer-mate`) containing a flawed `checked_shlw` (shift left word) function intended to guard a `<< 64` shift. The function's overflow threshold was mathematically incorrect (`0xFFFFFFFFFFFFFFFF << 192` instead of `1 << 192`). This allowed an attacker to pass an input $\geq 2^{192}$. When the `n << 64` shift executed, the value silently overflowed and truncated to a microscopically small numerator. Consequently, the protocol calculated that the attacker only needed to deposit 1 token to mint an astronomical amount ($\sim 10^{37}$) of liquidity pool shares, which they instantly redeemed to drain the DEX.

Mitigation Strategies: Defending against boundary breaches requires strict adherence to compiler safety features and rigorous mathematical verification.

- **Compiler Safeguards:** On the EVM, exclusively utilize Solidity $\geq 0.8.0$ and strictly

avoid the use of `unchecked` arithmetic blocks unless the logic is formally verified to be impervious to edge-case inputs.

- **Custom Library Verification:** Shared mathematical libraries (especially on non-EVM chains handling bitwise operations and fixed-point scaling) must undergo rigorous formal analysis and differential testing to ensure custom overflow guards do not contain flawed thresholds.
- **Boundary Fuzzing:** QA processes must implement property-based fuzz testing explicitly targeting extreme value ranges (minimum and maximum theoretical values for `u64`, `u128`, and `u256`) to observe protocol behavior near arithmetic boundaries.

4.10 SC10:2026 - Proxy & Upgradeability Vulnerabilities

Conceptual Framework: Because smart contracts are fundamentally immutable once deployed, developers utilize "Proxy" architectures to enable future protocol upgrades. This pattern separates the system into two components: a Proxy contract (which holds the user funds and state variables) and an Implementation or Logic contract (which contains the executable rules). A vulnerability here is analogous to a corporate structure where the operational headquarters (Logic) and the bank vault (Storage) are distinct entities; if the headquarters is left momentarily without a registered owner, a malicious actor can claim ownership, issue forged directives to the vault, or demolish the headquarters entirely.

Technical Anatomy & EVM Mechanics: Proxies function via the `delegatecall` opcode, executing the Logic contract's bytecode but applying all state changes to the Proxy's own storage. Because standard `constructor()` functions do not apply their state changes to the Proxy during a `delegatecall`, upgradeable contracts must rely on custom `initialize()` functions.

If developers fail to call `initialize()` atomically during deployment, the contract remains in an unowned, default state. Attackers actively monitor the mempool for such deployments. This vector was systematically exploited during the **Uninitialized Proxy Campaigns of 2025 (resulting in over \$10M in losses, including the \$1.55M Kinto Protocol hack)**. Attackers used automated bots to detect freshly deployed ERC1967 proxies that were not properly

initialized. They instantly initialized these proxies themselves, injecting a malicious logic implementation with a dormant backdoor. Months later, once the protocol had accrued significant Total Value Locked (TVL), the attackers activated the backdoor and drained the funds.

Additionally, if the underlying Logic contract itself is left uninitialized, an attacker can directly invoke its `initialize()` function, assume ownership, and execute the `selfdestruct` opcode. This obliterates the logic bytecode, leaving the Proxy contract permanently pointing to an empty address, thus "bricking" the protocol and freezing all user funds indefinitely.

Mitigation Strategies: Securing upgradeable architectures requires strict deployment hygiene and decentralized governance.

- **Atomic Initialization:** Proxies must be deployed and initialized in a single, atomic transaction to eliminate the mempool window where an attacker can hijack the initialization phase.
- **Implementation Locking:** Developers must explicitly invoke the `_disableInitializers()` mechanism within the constructor of the underlying Logic contract. This permanently prevents any external actor from initializing or hijacking the raw implementation contract.
- **Decentralized Upgrade Paths:** Upgrade execution (`upgradeTo()`) must never be controlled by a single Externally Owned Account (EOA). Upgrades should be gated behind Multi-Signature wallets and strict, on-chain Timelocks to allow the community to audit the new implementation before it goes live.

5 Vulnerability Detection Methodologies

This section defines the benchmark methodology, the detection logic of each paradigm and the evaluation criteria applied to the resulting outputs.

5.1 Methodology

The dataset is built on the Minimal Viable Exploit (MVE) principle, where each OWASP category is represented by one or two pairs of contracts, each Vulnerable contract has a syntactically equivalent Fixed counterpart that mitigates

the specific OWASP vector and changes nothing else. The dataset contains 30 Solidity contracts (15 Vulnerable + 15 Fixed) mapped to the OWASP Smart Contract Top 10. Pairing makes both true positives (on Vulnerable contracts) and false positives (on Fixed contracts) directly measurable on the same code surface.

Detection criteria. A finding is counted as a True Positive only when it identifies the specific OWASP vector instantiated by the MVE — for example, a reentrancy-eth finding on `MVE_08_Vuln.sol` counts; a generic low-level-call warning on the same contract does not. Any vulnerability meeting the per-tool severity thresholds below reported in a Fixed contract is classified as a False Positive. Detection criteria were normalized across tools to filter out non-critical noise:

- Static Analysis (Slither, Aderyn): High or Medium severity findings only.
- Symbolic Execution (Mythril): Any identified SWC (Smart Contract Weakness Classification) code.
- Fuzzing (Echidna, Medusa): Any failed invariant.
- LLM Review: Scores ≤ 4 on Vulnerable contracts; scores ≥ 6 on Fixed contracts.

Tool selection. Selection prioritized production-grade analyzers in active maintenance and broad industry use. Tools requiring incompatible environments were excluded — Manticore depends on pysha3 and has been unmaintained since 2022; SmartCheck requires Java 8 and breaks under modern JDKs (javax.xml.bind removal). Semgrep and Solhint are included as standard-tooling baselines: they represent the lowest-friction analyzers a Solidity developer encounters in a default CI pipeline and their detection rate measures what protection the standard toolchain offers without dedicated security tooling.

Reproducibility. All tool versions and configuration parameters are pinned in `config/versions.env` and per-tool configuration files. Table 1 summarizes the values used for the benchmark runs.

Table 1: Tool versions and key parameters.

Tool	Version	Key parameters
Slither	0.10.4	default detectors, JSON output
Aderyn	0.6.8	per-file scope (-i)
Semgrep	1.75.0	p/smart-contracts ruleset
Solhint	5.0.3	solhint:recommended
Mythril	0.24.8	execution timeout 60s
Halmos	0.2.1	default symbolic depth
Echidna	2.3.2	testLimit*=50000, seqLen*=100
Medusa	1.5.1	testLimit=50000, workers*=4
LLaMA-3.3-70B	via Groq	temperature=0.1, max_tokens=2048
Gemini 2.5 Flash	google-genai	temperature=0.1
Qwen3-32B	via Groq	temperature=0.1, max_tokens=4096

***testLimit**: randomized transactions per run; **seqLen**: max transaction sequence length; **workers**: parallel fuzzing processes.

Pipeline. The execution pipeline is sequential: static analysis (Slither, Aderyn, Semgrep, Solhint), symbolic and formal analysis (Mythril, Halmos), property-based fuzzing (Echidna, Medusa) and LLM-assisted review (LLaMA, Gemini, Qwen). Each stage writes outputs to a deterministic results directory; a unified report is generated by `scripts/build_report.py`.

5.2 Detection paradigms

Each paradigm has its own internal logic, false-positive profile and operational cost. The remainder of this section describes how each paradigm reaches a verdict and where its structural limits lie.

5.2.1 Static analysis: source-level pattern detection

Static analyzers read the Solidity source and flag suspicious patterns without executing the code. The paradigm splits cleanly into two subcategories that produce structurally different output and should not be aggregated when reporting recall.

Semantic analyzers (Slither, Aderyn) build a Solidity-specific intermediate representation of the contract and apply detectors that encode EVM-specific knowledge. Slither’s reentrancy-eth detector, for instance, recognizes the call-before-state-write pattern that defines CWE-841. Aderyn’s tx-origin detector recognizes authentication-by-origin as a phishing surface. The detectors know what they are looking for at the level of attack vector.

Pattern matchers and linters (Semgrep, Solhint) operate without domain knowledge. Semgrep matches generic syntactic templates from a community ruleset. Solhint enforces style conventions and a small set of surface-level secu-

rity rules. They are not vulnerability-aware in the same sense — they detect what their author wrote a rule for, not what the underlying exploit pattern requires.

5.2.2 Symbolic execution and formal verification: path-level reasoning

Both Mythril and Halmos reason about contract execution paths using SMT solvers, but they answer different questions and require different inputs from the user.

Mythril runs autonomously on EVM bytecode and checks each execution path against a fixed catalogue of vulnerability templates from the SWC registry. When the solver finds an input that drives the contract along a path violating one of these templates, Mythril reports a finding. The user provides only the contract and Mythril provides the templates.

Halmos does the opposite: the user provides the property to prove, written as a Solidity function prefixed `check_*`. Halmos then verifies that property over the entire symbolic input space. A successful execution does not simply mean that “no counterexample was found within the search frontier”, it is a formal proof that no input exists for which the property fails. The cost is that the user must author each property, but the benefit is that a proof, when obtained, is conclusive.

5.2.3 Property-based fuzzing: invariant testing through randomized execution

Echidna and Medusa generate random sequences of transactions and check, after each one, whether user-defined invariants still hold. The invariants are functions prefixed `echidna_` that return a boolean — for example, `echidna_balance_never_decreases()`. If the fuzzer finds a transaction sequence that breaks the invariant, it reports a minimized counterexample.

The two fuzzers share the same invariant language but differ in implementation. Echidna is single-process and written in Haskell; Medusa parallelizes workers natively in Go. Both are included in the benchmark because cross-validation matters: when Echidna and Medusa report identical findings on a contract, that result is unlikely to be specific to one fuzzer’s implementation.

5.2.4 LLM-based analysis: prompt-based audit

The three models receive the contract source through a single API call with a fixed audit prompt (Listing 1). The prompt asks for a structured response: vulnerability identification, severity classification and a single overall security score on a 1–10 scale (1 = critically vulnerable, 10 = perfectly secure). Temperature is set to 0.1 across all three models to reduce response variability across runs.

The score is extracted from the structured response field. For each model, three properties characterize performance on the benchmark:

- **Discrimination** ($\Delta = \text{avg}(\text{Fixed}) - \text{avg}(\text{Vuln})$): whether the model assigns lower scores to Vulnerable contracts and higher scores to Fixed variants on the same code surface.
- **False-positive rate on Fixed**: whether the model recognizes mitigation patterns (SafeERC20, TWAP oracles, signature nonces, `_disableInitializers`) rather than scoring on residual code complexity.
- **Output parseability**: whether the structured response format is preserved across all 30 contracts.

The empirical values are reported in Section 6.

```
1 You are an expert Solidity smart
  contract security auditor. Analyze
  the following smart contract and
  identify ALL security vulnerabilities
2
3 For each vulnerability found, report:
4 1. Vulnerability type (e.g., reentrancy
  , integer overflow, access control,
  etc.)
5 2. Severity: Critical / High / Medium /
  Low / Informational
6 3. Affected function and line number(s)
7 4. Brief description of the risk
8 5. Recommended fix (1-2 sentences)
9
10 Also provide at the end:
11 - Overall security score (1-10, where
  10 = perfectly secure, 1 = critically
  vulnerable)
12 - One-sentence summary of the most
  critical finding (or "No critical
  issues found")
13
14 Contract:
15 ```solidity
16 {code}
17 ```
```

Listing 1: Audit prompt sent to all three LLMs.

Implementation note: The Qwen3-32B invocation prepends the literal prefix `/no_think\n\n` to the prompt (see `scripts/llm_analyze.py`);

LLM evidence is treated as a semantic complement, not as execution-ground truth. The paradigm has structural limits — no direct execution of EVM paths, susceptibility to hallucinated findings, and dependence on training distribution and prompt framing — that prevent it from substituting for static, symbolic or fuzzing analysis.

Section 6 reports the empirical results of applying these paradigms to the 30-contract dataset.

6 Discussion

This section reports cross-tool measurements across all four paradigms, ordered as in 5.2. Section 6.1 summarizes per-tool results; 6.2 examines category-level behavior; 6.3 synthesizes the cross-paradigm picture.

6.1 Cross-tool analysis

Within the static paradigm, the two sub-paradigms behaved as predicted by 5.2. Semantic analyzers (Slither, Aderyn) detected vulnerable variants but flagged Fixed contracts indiscriminately, with Slither providing the strongest signal (7/15) and Aderyn covering fewer categories (3/15). Pattern matchers and linters (Semgrep, Solhint) produced no exploit-level findings — confirming that generic AST templates and style rules cannot substitute for vulnerability-aware detectors.

Mythril improved evidential quality where satisfiable paths were found (6/15 vulnerable cases, 10/15 FP on Fixed), with recurrent SWC-107 noise on reentrancy-adjacent Fixed contracts as the main source of false positives. Halmos demonstrated the formal verification paradigm in the two categories where specific properties were authored (SC01, SC09). In these cases, the tool provided absolute certainty: it produced a symbolic counterexample for each Vulnerable contract and a formal proof for each Fixed variant. The remaining 26 contracts are reported as non-applicable, as Halmos requires manual property specification `check_*` and does not operate as an autonomous scanner.

Fuzzing produced the cleanest behavior on Fixed contracts. Echidna and Medusa each de-

tected 10/15 vulnerable cases and maintained strong precision on non-vulnerable variants. Remaining false negatives were concentrated in two structurally constrained categories: SC05 (off-chain ECDSA signatures, which fuzzers cannot generate) and SC08 (cross-contract reentrancy callbacks, which require richer adversarial harnesses than the current setup provides).

All three LLMs successfully parsed all 30 contracts. LLaMA-3.3-70B achieved the strongest discrimination ($\Delta = +3.3$), with Vulnerable contracts consistently scoring in the 2–4 range and mitigated contracts in the 6–8 range. Qwen3-32B followed closely ($\Delta = +2.9$), with a slightly compressed Fixed range (avg 6.2 vs LLaMA’s 6.8) suggesting more conservative scoring of mitigated variants.

Both models distinguished syntactic-vulnerability categories (SC01, SC06, SC09) from semantic-logic categories (SC02, SC03, SC04, SC07). The pattern reflects a structural limit of source-only LLM analysis: the models score on what the code says, not on what the code does after execution. Syntactic vulnerabilities (a missing onlyOwner, an unchecked block, a low-level call with ignored return value) are visible in the token stream and the models flag them reliably. Logic vulnerabilities are not. A function that performs token.transfer(user, amount) without a corresponding balances[user] -= amount line reads as a clean, well-formed transfer; the bug — that the user’s balance is never debited — exists only as a property of the post-execution state, which the LLM does not reason about. On contracts of this kind in the SC02 family, LLaMA and Qwen assigned low scores (LLaMA 4/10, Qwen 3/10), correctly flagging the drainability of the contract.

Gemini 2.5 Flash failed to discriminate between Vulnerable and Fixed contracts, with $\Delta = +1.4$. The cause is not under-detection but systematic over-flagging of mitigated contracts. Nine Fixed variants received scores $\leq 3/10$ despite implementing the prescribed mitigations: SC03-Fixed (TWAP oracle), SC04-Fixed, SC05-Fixed, SC05-Fixed2, SC06-Fixed (SafeERC20 + CEI), SC07-Fixed, SC09-Fixed, SC09-Fixed2, and SC10-Fixed. Gemini appears to score on residual code complexity (presence of low-level calls, presence of arithmetic blocks, presence of delegatecall) rather than on whether the primary OWASP vector is mitigated — a precision problem, not a recall problem. On SC09-Vuln, Gem-

Table 2: Compact quantitative summary from the unified benchmark report.

Type	SubType	Tool	TP*	FP**	Main tations	Limita- tions
Static	Semantic	Slither	7/15	15/15	Generic	patterns on all Fixed
	Semantic	Aderyn	3/15	15/15	ETH-address	noise on all Fixed
	Patt. match	Semgrep	0/15	0/15	No EVM	semantics
	Lint/style	Solhint	0/15	0/15	Surface	security rules only
Symbolic	-	Mythril	6/15	10/15	SWC-107	noise on reentrancy
	-	Halmos	2/2	0/2	Needs	check_*
Fuzzing	-	Echidna	10/15	0/15	FN:ECDSA(SC05),	callbacks(SC08)
	-	Medusa	10/15	0/15	Same	structural blind spots

* TP = Vulnerable contracts where a finding identified the specific OWASP vector instantiated by the MVE, per the criteria defined in 5.1.

** FP = Fixed contracts that received any finding meeting the severity thresholds defined in 5.1.

Table 3: LLM security score averages across Vuln and Fixed contracts.*

Model	Vuln avg	Fixed avg	Δ	Parsed
LLaMA-3.3-70B	3.5	6.8	+3.3	30/30
Gemini 2.5 Flash	2.5	3.9	+1.4	30/30
Qwen3-32B	3.3	6.2	+2.9	30/30

* Scale: 1 = critically vulnerable, 10 = perfectly secure. $\Delta = \text{avg(Fixed)} - \text{avg(Vuln)}$; higher values indicate stronger discrimination between vulnerable and mitigated variants. Parsed = contracts from which a numeric score was successfully extracted.

ini assigned a score of 6/10 — within the band reserved for mitigated contracts (≥ 6 per §5.1) despite the `unchecked { totalSupply += amount }` block. This is the only case where Gemini failed to place a Vulnerable contract below the detection threshold; the likely cause is that Gemini relied on the Solidity $\geq 0.8.0$ compiler default without re-evaluating the `unchecked` exception.

The practical implication for an audit pipeline is straightforward: LLaMA-3.3-70B is the safest default for automated triage (highest Δ , fully parseable output, no critical false positives); Qwen3-32B reaches comparable discrimination ($\Delta +2.9$) and is the natural fallback when LLaMA is unavailable. Gemini is unsuitable for Vuln/Fixed scoring without prompt recalibration but may remain useful for verbose qualitative analysis where over-flagging is tolerable.

6.2 Category-level behavior

Detection quality varied by vulnerability family:

- **SC01/SC06** (access control, unchecked calls) were detectable by multiple

paradigms, but only fuzzers achieved this without false positives on the corresponding Fixed variants. Slither and Mythril detected the vulnerabilities but flagged the Fixed counterparts indiscriminately, replicating the precision pattern of §6.1 at category level.

- **SC09** (integer arithmetic) showed split behavior: Mythril and both fuzzers detected vulnerable variants; static tools produced non-discriminative output (findings on both Vuln and Fixed). Halmos provided formal proof on the SC09 pair where `check_*` properties were authored, and was the only paradigm to produce conclusive evidence rather than detection alone.
- **SC10** (uninitialized proxy) was better captured by fuzzing than by static or symbolic tools — initialization race conditions are runtime properties that source-level pattern matching cannot reach.
- **SC05** and **SC08** exposed structural harness limits: off-chain signatures cannot be generated by property-based fuzzers, and cross-contract callbacks require richer adversarial setups than the current harnesses provide. The false negatives on these categories are limitations of harness expressiveness, not of the fuzzers themselves.

6.3 Cross-paradigm complementarity

The benchmark confirms the value of paradigm composition. Static tools are effective first-pass filters despite high FP rates; symbolic analysis provides path-level evidence when tractable; fuzzing validates runtime invariants with high practical precision and just one FP on Fixed variants; LLMs support semantic triage without replacing executable verification; when formal properties are authored, Halmos provides conclusive proof rather than probabilistic detection. No single paradigm dominated all categories. Best coverage emerged from paradigm composition: the three-stage pipeline of §7 (Slither → Mythril → Echidna) covers the OWASP surface where Halmos’s `check_*` properties were not authored, with each stage compensating for the precision and recall failure modes of the previous one.

7 Conclusion

This work delivered an empirical benchmark of smart contract security tooling over 30 Solidity contracts organized as Vulnerable/Fixed MVE pairs across the OWASP Smart Contract Top 10. The Vuln/Fixed design exposed an asymmetry that aggregated benchmarks tend to hide. Static and symbolic tools achieved lower recall than fuzzers (Slither 7/15, Mythril 6/15, against 10/15 for both Echidna and Medusa) and at substantially higher false-positive cost (15/15 and 10/15 respectively, against 0/15 for fuzzing). Fuzzers dominated this corpus on both axes; the question for an audit pipeline is therefore not whether to use them, but how to compose them with paradigms that capture what they cannot.

The data support a minimal three-stage audit pipeline: Slither → Mythril → Echidna. Slither operates as a fast first-pass filter on suspicious patterns; Mythril provides path-level confirmation on flagged functions through symbolic execution over EVM bytecode; Echidna validates that residual invariants resist tens of thousands of randomized transactions. Halmos extends this stack with formal proof on categories where `check_*` properties can be authored — demonstrated here on SC01 and SC09, where the property authored against the Vulnerable contract produced a symbolic counterexample and the equivalent property against the Fixed contract was proven over the entire symbolic input space. Unlike a passing fuzzer or a silent static analyzer, this is conclusive evidence that no input exists for which the property fails.

Aderyn contributed 3/15 detections at the same 15/15 false-positive rate as Slither, with positives that largely overlap Slither’s coverage — limited additive value beyond the stronger semantic analyzer. Semgrep and Solhint produced no exploit-level findings and serve only as standard-tooling baselines: their 0/15 detection rate measures what protection the default Solidity CI pipeline offers without dedicated security tooling. LLM review is useful for triage and qualitative explanation, but the gap between the best and worst calibration in this benchmark ($\Delta +3.3$ for LLaMA-3.3-70B against $+1.4$ for Gemini 2.5 Flash) shows that the paradigm is sensitive enough to model and prompt that it cannot serve as a verdict layer in an automated pipeline. The structural absence of executable grounding remains the deeper reason.

Two limitations bound the empirical reach of

this benchmark. First, the MVE design produces compact, single-vector contracts: each Fixed/Vulnerable pair isolates one OWASP category, which simplifies attribution but excludes interaction effects between vectors that real-world contracts exhibit. Second, the fuzzing harness for SC05 was intentionally minimal — generic invariant functions without ECDSA generation or attacker contracts — while the SC08 harnesses do include dedicated attacker contracts; the false negatives observed on SC05 therefore reflect harness expressiveness, not a structural limit of property-based fuzzing itself. Three extensions would broaden the result. Authoring `check_*` properties for the remaining eight OWASP categories would convert Halmos from per-category formal proof to a coverage-wide guarantee. Replacing or augmenting the synthetic MVE corpus with real-world contracts would test whether tool behavior generalizes to production code. Calibrating the LLM prompt against a dedicated mitigation-recognition test set could close the precision gap observed on Gemini, likely a prompt-engineering problem rather than a model-capacity limitation, since LLaMA and Qwen achieved strong discrimination with the same prompt.

References

- [1] D. Liu, J. Zhang, Y. Wang, H. Shen, Z. Zhang, and T. Ye, “Blockchain smart contract security: Threats and mitigation strategies in a lifecycle perspective,” *ACM*, 2025. DOI: 10.1145/3769013. [Online]. Available: <https://dl.acm.org/doi/full/10.1145/3769013>.
- [2] M. Ortu, G. Ibba, G. Destefanis, C. Conversano, and R. Tonelli, “Taxonomic insights into ethereum smart contracts by linking application categories to security vulnerabilities,” *Scientific Reports (Nature)*, 2024. DOI: 10.1038/s41598-024-73454-0. [Online]. Available: <https://www.nature.com/articles/s41598-024-73454-0>.
- [3] OWASP Foundation, *Owasp smart contract top 10*, Acedido em: 26-Fev-2026, 2024. [Online]. Available: <https://owasp.org/www-project-smart-contract-top-10/>.
- [4] H. Zhou, A. M. Fard, and A. Makanju, “The state of ethereum smart contracts security: Vulnerabilities, countermeasures, and tool support,” *MDPI*, vol. 2, no. 2, p. 19, 2022. [Online]. Available: <https://www.mdpi.com/2624-800X/2/2/19>.
- [5] S. S. Kushwaha, S. Joshi, D. Singh, M. Kaur, and H.-N. Lee, “Systematic review of security vulnerabilities in ethereum blockchain smart contract,” in *IEEE*, 2021. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9667515>.
- [6] T. Jiao, Z. Xu, M. Qi, S. Wen, Y. Xiang, and G. Nan, “A survey of ethereum smart contract security: Attacks and detection,” *ACM Computing Surveys*, 2024. DOI: 10.1145/3643895. [Online]. Available: <https://dl.acm.org/doi/full/10.1145/3643895>.
- [7] J. Zhang, Y. Shen, J. Chen, *et al.*, “Demystifying and detecting cryptographic defects in ethereum smart contracts,” in *IEEE*, 2024. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11029839/>.
- [8] P. Qian, R. Cao, Z. Liu, *et al.*, “Comprehensive review of smart contract and defi security: Attack, vulnerability detection, and automated repair,” *ScienceDirect*, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0957417425020500>.